PPT 讲解稿

Attention Is All You Need

Attention Is All You Need

第 1 页：Attention Is All You Need

各位老师、同学们好，今天我要为大家介绍的这篇论文是深度学习领域具有里程碑意义的著作——由 Google 团队 Ashish Vaswani 等人于 2017 年发表的《Attention Is All You Need》。这篇论文首次提出了完全基于注意力机制的 Transformer 架构，彻底改变了序列建模和 transduction 任务的研究范式。

在开始介绍之前，让我们了解一下这支明星团队的研究背景。作者们来自 Google Brain 和 Google Research，包括提出了自注意力替代 RNN 想法的 Jakob Uszkoreit、负责核心架构设计的 Ashish Vaswani 和 Illia、以及提出缩放点积注意力、多头注意力和位置编码等关键技术的 Noam Shazeer 等人。这篇论文发表于神经信息处理系统的顶级会议 NIPS 2017 上，至今已被引用超过十万次，是自然语言处理领域引用量最高的论文之一。

接下来的演讲中，我将从研究动机出发，详细解读 Transformer 的核心技术，并在最后展示其实验结果和广泛影响。

可能被问到：为什么取名"Attention Is All You Need"？这是对 Beatles 乐队名曲"All You Need Is Love" 的致敬，表明作者认为注意力机制是序列建模的核心。

第 2 页：研究背景与动机

首先让我们回顾一下序列建模领域的研究背景。在 Transformer 出现之前，循环神经网络及其变体——包括长短期记忆网络 LSTM 和门控循环单元 GRU——是处理序列数据的绝对主流方法。这些模型在语言建模、机器翻译等序列到序列的 transduction 任务中取得了显著进展。

然而，RNN 模型存在一个根本性的限制：其顺序计算特性。RNN 在处理序列时，必须沿着符号位置逐步进行计算，每个时间步的隐藏状态依赖于前一时刻的隐藏状态。这种固有的串行性质使得训练时无法在样本内部进行并行化，当序列长度增加时，内存约束会严重影响批处理能力。尽管研究者们尝试通过分解技巧和条件计算来改善效率，但顺序计算的根本限制始终无法突破。

与此同时，注意力机制作为一种允许建模任意位置间依赖关系的技术，已经被广泛应用于序列模型中。Bahdanau 等人提出的对齐机制让模型能够在生成每个输出时关注输入的相关部分。但这些注意力机制通常仍需与 RNN 结合使用。

正是在这样的背景下，一个核心问题浮现出来：能否完全摒弃 RNN 和卷积，仅用注意力机制来实现高性能的序列 transduction 模型？这正是 Transformer 要回答的问题。

可能被问到：为什么选择 machine translation 作为主要评估任务？因为当时 NMT 是 NLP 领域最重要的 benchmark，能够充分验证模型能力。

第 3 页：相关工作

在正式介绍 Transformer 之前，我们需要了解一些相关的技术背景。这些相关工作从不同角度为 Transformer 的设计提供了启发。

第一类相关工作是利用卷积神经网络作为基本构建块的模型，包括 Neural GPU、ByteNet 和 ConvS2S 等。这些模型通过卷积操作能够并行计算所有输入和输出位置的隐藏表示，大大提高了计算效率。然而，这些卷积模型存在一个显著问题：建立两个任意位置之间依赖关系所需的运算次数随距离线性增长（ConvS2S）或对数增长（ByteNet）。这使得学习长距离依赖变得困难。

第二类相关工作是自注意力机制，即关联同一序列中不同位置以计算序列表示的技术。自注意力已成功应用于阅读理解、抽象摘要、文本蕴含等任务，并在学习任务无关的句子表示方面展现出潜力。

第三类相关工作是端到端记忆网络，这类模型基于循环注意力机制而非序列对齐的循环结构，在简单语言的问答和语言建模任务上表现良好。

值得强调的是，据作者所知，Transformer 是第一个完全依赖自注意力来计算输入和输出表示的 transduction 模型，既不使用序列对齐的 RNN 也不使用卷积。这一设计选择使 Transformer 能够在保持高质量的同时实现前所未有的并行化程度。接下来的章节中，我们将详细探讨 Transformer 的具体架构。

可能被问到：为什么 Transformer 比 CNN 更适合建模长距离依赖？因为注意力机制允许任意位置直接交互，不受卷积核大小的限制。

第 4 页：Transformer 模型架构

现在让我们深入了解 Transformer 的整体架构。Transformer 采用了广泛使用的 Encoder-Decoder 结构，这是大多数竞争性神经序列 transduction 模型的标准范式。

具体来说，Encoder 的作用是将输入符号序列（x , ..., x ）映射为一个连续的表示序列 z = (z , ..., z)。这个连续表示编码了输入序列的语义信息，是后续解码的基础。

Decoder 则在此基础上，自回归地生成输出符号序列（y , ..., y)。所谓自回归，是指模型在每个时间步生成下一个符号时，会将已经生成的所有符号作为额外输入。这种方式确保了生成的有序性和一致性。

Transformer 的独特之处在于，它完全基于堆叠的自注意力层和逐位置的全连接层来构建 Encoder 和 Decoder，没有使用任何循环或卷积操作。Encoder 和 Decoder 各由 N=6 个相同的层堆叠而成。每个层包含两个主要子层：一个是多头自注意力机制，另一个是位置-wise 的全连接前馈网络。

这种架构设计带来了显著的优势：由于注意力机制允许所有位置并行交互，Transformer 可以充分利用 GPU 的并行计算能力，大幅缩短训练时间。接下来，我们将逐一详解这些核心组件。

可能被问到：为什么选择 N=6 层？这是一个经验性的选择，论文通过实验发现 6 层能够在性能和计算成本之间取得良好平衡。

第 5 页：注意力机制 - Scaled Dot-Product Attention

让我们首先介绍 Transformer 中使用的基础注意力机制——缩放点积注意力，这是整个模型的核心组件。

注意力函数可以描述为将一个查询（Query）和一组键值对（Key-Value pairs）映射到输出的过程。在 Transformer 中，输入包括维度为 d 的查询 Q 和键 K，以及维度为 d 的值 V。输出则是值的加权和，权重由查询与相应键的兼容性函数决定。

具体计算公式为：Attention(Q, K, V) = softmax(QK /√d)V。这个公式的直觉是：首先计算查询与所有键的点积，得到原始注意力权重；然后除以 √d 进行缩放，防止点积值过大；最后通过 softmax 归一化得到真正的注意力权重，再用这些权重对值进行加权求和。

在实际操作中，我们通常将多个查询打包成矩阵 Q，同时将键和值打包成矩阵 K 和 V，通过一次矩阵运算就能并行计算所有查询的注意力输出。

与加性注意力相比，点积注意力具有显著的速度和空间优势，可以利用高度优化的矩阵乘法代码实现。但为什么需要缩放因子 1/√d 呢？作者通过分析发现，当 d 较大时，点积的值往往很大，这会将 softmax 函数推入极小梯度区域，导致训练困难。缩放操作有效地缓解了这个问题。

可能被问到：为什么用 softmax 而不是其他归一化函数？softmax 能够产生稀疏的注意力分布，且其梯度在反向传播时具有良好的数学性质。

第 6 页：注意力机制 - Multi-Head Attention

单一注意力机制虽然有效，但作者发现将查询、键和值投影到多个不同的子空间能够进一步提升模型性能，这就引出了多头注意力机制。

多头注意力的核心思想是：不是使用单一的高维注意力函数，而是将 Q、K、V 分别投影 h 次到不同的低维子空间，在每个子空间并行执行注意力计算，然后将所有头的输出拼接起来，再进行一次线性变换得到最终输出。

具体公式为：MultiHead(Q, K, V) = Concat(head , ..., head)W ，其中每个 head = Attention(QW , KW , VW)。

在论文的实验中，使用 h=8 个并行注意力头，每个头的维度设为 d = d = d d /h = 64。由于每个头的维度较小，总计算成本与单头全维度注意力相近，但效果显著提升。

多头注意力允许模型同时关注来自不同表示子空间的信息。这种设计的直觉是：不同的注意力头可能专注于捕捉不同类型的关系，比如语法结构、语义关联或位置关系等。在后面的可视化分析中，我们将看到不同注意力头确实表现出了这种专业化的趋势。

Transformer 以三种方式应用多头注意力：编码器-解码器注意力层（允许解码器关注输入序列）、编码器自注意力层（允许每个位置关注编码器前一层所有位置）、以及解码器自注意力层

（允许关注当前位置及之前的位置，但通过掩码防止看到未来信息）。

可能被问到：为什么选择 h=8 和 d=64？作者通过实验发现这个配置在翻译任务上效果最佳，过多或过少的头数都会导致性能下降。

第 7 页：位置编码与前馈网络

除了注意力机制，Transformer 还需要另外两个重要组件来完整实现序列建模能力：位置编码和逐位置前馈网络。

由于 Transformer 完全基于注意力机制，不包含任何循环或卷积操作，因此模型本身无法感知输入序列中符号的绝对或相对位置信息。为了解决这个问题，论文采用向输入嵌入向量中添加位置编码的方式注入位置信息。位置编码与嵌入向量具有相同的维度 d d = 512，因此两者可以直接相加。

论文使用了正弦和余弦函数来生成位置编码：对于偶数维度使用 sin 函数，奇数维度使用 cos 函数。这种设计的精妙之处在于，对于任意固定的偏移量 k，位置编码 PE 可以表示为 PE 的线性函数。这一特性使得模型能够容易地学习关注相对位置关系。

另一个关键组件是逐位置前馈网络。这个网络对每个位置独立应用相同的双层全连接变换，中间层使用 ReLU 激活函数。公式为：FFN(x) = max(0, xW + b)W + b 。其中输入输出维度为 d d = 512，内层维度 dff = 2048。这个前馈网络可以看作两个核大小为 1 的一维卷积。

虽然不同位置共享前馈网络的参数，但由于每层都有独立的权重矩阵，不同层之间实际上是不同的。这两个子层——多头注意力和前馈网络——都使用了残差连接和层归一化来促进梯度流动和稳定训练。

可能被问到：为什么选择正弦/余弦编码而不是可学习的位置嵌入？正弦编码允许模型处理任意长度的序列，而可学习嵌入受限于训练时的最大长度。

第 8 页：机器翻译实验结果

现在让我们来看看 Transformer 的实际表现。论文在两个标准的机器翻译基准上进行了实验。

首先是 WMT 2014 英德翻译任务。Transformer 的大模型在这个任务上达到了 28.4 BLEU 的分数，比此前最佳模型（包括集成模型）还要高出 2 个多 BLEU，这是一个显著的提升。建立这样的新纪录充分证明了 Transformer 架构的优越性。

其次是 WMT 2014 英法翻译任务。Transformer 大模型达到了 41.8 BLEU 的分数，成为新的单模型最高水平。更令人印象深刻的是，这只花了 3.5 天的训练时间，使用 8 块 NVIDIA P100 GPU。相比之下，之前的最佳模型需要数周甚至更长的训练时间。

在训练配置方面，基础模型使用 100K 步训练，耗时约 12 小时；大模型使用 300K 步训练，耗时 3.5 天。每个训练批次包含约 25000 个源 tokens 和 25000 个目标 tokens。模型使用 Adam 优化器配合学习率预热策略训练，使用标签平滑化和 Dropout 正则化防止过拟合。

这些结果不仅展示了 Transformer 在翻译质量上的优势，更重要的是证明了基于注意力机制的架构在训练效率上的巨大潜力。

可能被问到：为什么 BLEU 分数提升 2 分被认为是重大突破？因为在机器翻译领域，BLEU 分数的提升非常困难，尤其是超越集成模型。

第 9 页：性能对比分析

让我们更详细地比较 Transformer 与其他模型的性能差异。

在英德翻译任务上，Transformer 大模型达到的 28.4 BLEU 远超此前最佳结果 26.3 BLEU（由 GNMT+RL 集成模型实现）。这意味着 Transformer 不仅超越了单一模型，还超越了当时最好的集成模型。

在英法翻译任务上，Transformer 大模型达到 41.8 BLEU，超越了此前最佳的 ConvS2S 集成模型的 41.29 BLEU。考虑到英语和法语在语言学上更为接近，翻译难度相对较低，所以绝对分数更高，但超越集成模型仍然意义重大。

更重要的是训练成本的对比。以 FLOPs（浮点运算次数）来衡量，Transformer 大模型训练英德翻译需要约 2.3×10^{1} 次浮点运算，而 GNMT 模型需要约 1.4×10^{2} 次。这意味着 Transformer 的训练成本降低了约六倍，同时获得了更好的翻译质量。

从另一个角度看，基础 Transformer 模型在仅 12 小时的训练后就超过了大多数先前发布的模型，这证明了纯注意力架构的高效性。这种高效性源于 Transformer 能够充分利用 GPU 的并行计算能力，因为注意力机制的核心运算是可以高度并行化的矩阵操作。

这些对比数据清晰地表明，Transformer 不仅在翻译质量上达到了新高度，而且在计算效率上也实现了质的飞跃。

可能被问到：BLEU 分数的局限性是什么？BLEU 是基于 n-gram 重叠的指标，可能无法完全捕捉翻译的语义准确性和流畅性。

第 10 页：注意力可视化分析

一个令人惊喜的发现是，Transformer 的注意力头展现出了可解释的结构化行为，这暗示模型确实学习到了语言的语法和语义结构。

第一个例子展示了编码器第 5 层的注意力如何捕捉长距离依赖关系。考虑句子"The verb 'making' needs to be connected to 'more difficult'"——这是典型的动词与其补语之间的依存关系。在可视化中，我们看到注意力头准确地将"making" 这个词与其远距离的补语"more difficult" 关联起来。这种能力对于准确理解句子含义至关重要。

第二个例子展示了注意力头在回指解析（anaphora resolution）任务中的作用。回指是指用代词指代前面出现的名词或实体。图中显示，注意力头能够将代词"its" 与其指代对象"the Law" 和"the application" 准确关联。这种能力表明模型理解了代词与先行词之间的语义关系。

更有趣的是，不同的注意力头似乎专门从事不同的任务。一些头专注于句法结构，另一些专注于语义关联，还有一些专注于位置关系。这种分工与合作的方式，与人类语言处理中的模块化

假设有着有趣的相似性。

这些可视化结果不仅增强了我们对模型的理解，也证明了纯注意力机制确实能够学习到语言中的复杂结构关系。

可能被问到：注意力权重是否总是可解释的？并非总是如此。有些注意力头可能学习到不太明显的模式，或者参与多种复杂的关系建模。

第 11 页：模型变体与消融实验

为了验证 Transformer 各组件的重要性，论文进行了一系列系统的消融实验。

首先是关于注意力头数和维度配置的实验（表 3 行 A）。作者保持总体计算量不变，改变头数和维度组合。实验结果表明，8 头配合 d =d =64 的配置效果最佳。单头注意力性能下降 0.9 BLEU，头数过多也会导致质量下降。这说明适当的注意力头数量和维度对于平衡不同子空间的信息至关重要。

关于键维度 d 的实验（行 B）揭示了一个重要发现：减小 d 会损害模型质量。这表明简单的点积兼容性函数在确定相关性方面存在一定局限，可能需要更复杂的兼容性度量来提升性能。这也暗示了为什么加性注意力在某些情况下仍然有效。

层数和大小的实验（行 C）验证了”大模型更好”的普遍规律。从 6 层减少到 4 层会导致性能明显下降，而将模型扩展到 dff=4096 和 dmodel=1024 则能显著提升质量。

Dropout 的实验（行 D）表明，Pdrop=0.1 的 dropout 率能够有效防止过拟合，移除 dropout 或增加 dropout 率都会导致性能下降。

最后，位置编码的实验（行 E）显示，可学习的 positional embedding 与论文使用的正弦编码产生了几乎相同的结果。基于这个发现，论文选择保留正弦编码，因为它可能允许模型外推到比训练时更长的序列。

可能被问到：为什么不使用更复杂的注意力机制？简单性是 Transformer 的设计哲学之一，简单的点积注意力加上多头机制已经足够强大。

第 12 页：泛化能力：成分句法分析

为了验证 Transformer 不仅仅适用于机器翻译任务，论文将其应用于英语成分句法分析这个完全不同的 NLP 任务。

成分句法分析（Constituency Parsing）是将句子分解为其组成部分的树结构的过程，这些成分形成语法单元。这项任务对输出有强结构性约束，且输出序列通常比输入序列更长，这些特点与机器翻译有显著不同。

实验在宾夕法尼亚树库的华尔街日报部分进行。在仅使用 WSJ 训练数据（约 40K 句子）的设置下，4 层 Transformer 模型达到了 91.3 的 F1 分数，超越了著名的 BerkeleyParser。这是一个引人注目的结果，因为 RNN 序列到序列模型此前无法在小数据场景下达达到最先进水平。

在半监督设置下，利用约 17M 句子的大型语料库进行训练，Transformer 达到了 92.7 的 F1 分数，接近最佳 RNN Grammar 模型的 93.3 分数。

值得注意的是，这些实验几乎使用了与翻译模型相同的超参数配置，没有进行任何任务特定的调优。这强有力地证明了 Transformer 架构具有良好的泛化能力，能够有效迁移到其他 NLP 任务。

可能被问到：为什么 Transformer 在结构化输出任务上表现良好？可能是因为注意力机制能够灵活地建模任意位置间的关系，适应不同的输出结构。

第 13 页：总结与展望

让我们来总结这篇论文的核心贡献和未来展望。

Transformer 的核心贡献可以概括为三点：第一，它提出了完全基于注意力机制的新架构，首次完全摒弃了 RNN 和卷积操作；第二，在机器翻译任务上，Transformer 实现了新的最高水平，同时训练速度提升了数倍；第三，通过成分句法分析实验，证明了纯注意力模型具有良好的泛化能力。

从更宏观的角度看，Transformer 的成功启发了一个新的研究范式。在此之后，大量基于 Transformer 的模型被提出，包括 BERT、GPT 系列、T5 等，它们在自然语言处理的各个任务上都取得了突破性进展。可以说，Transformer 开启了 NLP 领域的”预训练-微调”时代。

关于未来研究方向，论文提到了几个有前景的领域：首先是扩展到文本以外的其他模态，如图像、音频和视频处理；其次是研究局部受限的注意力机制，以高效处理更大的输入和输出；最后是使生成过程更少依赖顺序性，进一步提升并行效率。

这篇论文的代码已公开发布在 GitHub 的 tensor2tensor 仓库中，为后续研究提供了便利。

可能被问到：Transformer 的局限性是什么？主要局限性包括：注意力计算的二次方复杂度使得长序列处理困难，且模型缺乏对任务的理解而仅是统计模式匹配。

第 14 页：参考文献

最后，让我简要介绍这篇论文的核心参考文献，这些文献共同构成了 Transformer 设计的理论基础和技术背景。

首先是 Bahdanau 等人的工作，这是首次将注意力机制引入神经机器翻译，为后续研究奠定了基础。其次是 Gehring 等人的 ConvS2S，它展示了卷积序列到序列学习的潜力，尽管后来被 Transformer 超越。Hochreiter 和 Schmidreiter 的 LSTM 论文是深度学习处理序列数据的里程碑式工作，为理解循环网络提供了理论基础。Sutskever 等人的 Seq2Seq with Neural Networks 展示了端到端学习序列映射的可能性。最后是 Google 的 NMT 系统，它代表了当时最先进的工业级翻译系统。

这些参考文献涵盖了序列建模领域从循环网络到注意力机制的发展脉络，反映了 Transformer 站在前人肩膀上的创新过程。

感谢各位的聆听！如果大家有任何问题，我很乐意进行讨论。